

AI ASSISTED CODING

NAME : T .SAI VARUN

ROLLNO:2403A510C0

BATCH : 05

ASSIGNMENT : 17.1

#TASK-1

PROMPT :

Write a python program to Clean raw social media posts dataset.

Instructions:

- Remove stopwords, punctuation, and special symbols from post text.
- Handle missing values in likes and shares columns.
- Convert timestamp to datetime and extract features (hour, weekday).
- Detect and remove spam/duplicate posts.

CODE :

```
assignment.py > ...
1  # assignment.py
2  import pandas as pd
3  import numpy as np
4  import re
5  import nltk
6  from nltk.corpus import stopwords
7  from datetime import datetime
8
9  # Download stopwords (run once)
10 nltk.download('stopwords')
11
12 # -----
13 # Create sample dataset
14 # -----
15 data = {
16     'post_text': [
17         'I love this place!!!',
18         'New phone arrived today!',
19         'Feeling happy 😊',
20         'I love this place!!!',
21         'Good morning everyone!!!'
22     ],
23     'likes': [120, 89, 45, 120, np.nan],
24     'shares': [15, np.nan, 5, 15, 2],
25     'timestamp': [
26         '2025-10-25 18:45:00',
27         '2025-10-26 09:30:00',
28         '2025-10-26 12:15:00',
29         '2025-10-25 18:45:00',
30         '2025-10-27 08:00:00'
31     ]
32 }
```

```

assignment.py > ...
34 df = pd.DataFrame(data)
35
36 # -----
37 # Clean text
38 # -----
39 stop_words = set(stopwords.words('english'))
40
41 def clean_text(text):
42     if pd.isnull(text):
43         return ""
44     text = re.sub(r'^A-Za-z0-9\s', '', text) # remove punctuation
45     words = [word.lower() for word in text.split() if word.lower() not in stop_words]
46     return " ".join(words)
47
48 df['clean_post'] = df['post_text'].apply(clean_text)
49
50 # -----
51 # Handle missing values
52 # -----
53 df['likes'] = df['likes'].fillna(0)
54 df['shares'] = df['shares'].fillna(0)
55
56 # -----
57 # Extract time features
58 # -----
59 df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')
60 df['hour'] = df['timestamp'].dt.hour
61 df['weekday'] = df['timestamp'].dt.day_name()
62
63 # -----
64 # Remove duplicates/spam
65 # -----

```

```

65 # -----
66 df = df.drop_duplicates(subset=['clean_post'])
67 df = df[df['clean_post'].str.split().str.len() > 2]
68
69 # -----
70 # Save cleaned data
71 # -----
72 cleaned_df = df[['clean_post', 'likes', 'shares', 'hour', 'weekday']]
73 cleaned_df.to_csv("social_media_cleaned.csv", index=False)
74
75 print("✅ Cleaning complete! Saved as 'social_media_cleaned.csv'")
76 print(cleaned_df)
77

```

social_media_cleaned.csv

```

1 clean_post,likes,shares,hour,weekday
2 new phone arrived today,89.0,0.0,9,Sunday
3 good morning everyone,0.0,2.0,8,Monday
4

```

OUTPUT :

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "C:/Users/keerthi priya/Desktop/ai lab/as
signment.py"
[nltk_data] Downloading package stopwords to C:\Users\keerthi
[nltk_data] priya\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[✓] Cleaning complete! Saved as 'social_media_cleaned.csv'
      clean_post likes shares hour weekday
1 new phone arrived today 89.0 0.0 9 Sunday
4 good morning everyone 0.0 2.0 8 Monday
PS C:\Users\keerthi priya\Desktop\ai lab>
```

OBSERVATION :

1. Modules used are pandas , re , numpy , nltk and datetime for data clean task
2. nltk stopwords downloaded
3. Cleaned the text by removing punctuation, symbols, and stopwords.
4. Handled missing values by replacing empty likes/shares with 0.
5. Saved the cleaned data into a new file named social_media_cleaned.csv.

#TASK-2

PROMPT :

Write a python program to Preprocess a stock market dataset.

Instructions:

- Handle missing values in closing_price and volume.
 - Create lag features (1-day, 7-day returns).
 - Normalize volume column using log-scaling. - Detect outliers in closing_price using IQR method
- CODE :

ASSIGNMENT2.py > ...

```
1  # Task 2 - Financial Data Preprocessing
2
3  import pandas as pd
4  import numpy as np
5
6
7  # Step 1: Create sample stock market dataset
8  # -----
9  data = {
10     'date': pd.date_range(start='2025-10-01', periods=10, freq='D'),
11     'closing_price': [150, 152, np.nan, 155, 160, 300, 162, 158, np.nan, 159],
12     'volume': [1000, 1050, 980, np.nan, 1200, 5000, 1150, np.nan, 1100, 1080]
13 }
14
15 df = pd.DataFrame(data)
16
17 # -----
18 # Step 2: Handle missing values
19 # -----
20 # Fill missing closing_price with previous value (forward fill)
21 df['closing_price'] = df['closing_price'].fillna(method='ffill')
22
23 # Fill missing volume with mean value
24 df['volume'] = df['volume'].fillna(df['volume'].mean())
25
26 # -----
27 # Step 3: Create lag features (returns)
28 # -----
29 # 1-day return = (today - yesterday) / yesterday
30 df['return_1d'] = df['closing_price'].pct_change(1)
31
32 # 7-day return = (today - price 7 days ago) / price 7 days ago
33 df['return_7d'] = df['closing_price'].pct_change(7)
34
35 # -----
36 # Step 4: Normalize volume using log-scaling
37 # -----
```

Ln 19, Col 30 S

```

35 # -----
36 # Step 4: Normalize volume using log-scaling
37 # -----
38 df['volume_log'] = np.log1p(df['volume']) # log(1 + volume)
39
40 # -----
41 # Step 5: Detect outliers in closing_price using IQR
42 # -----
43 Q1 = df['closing_price'].quantile(0.25)
44 Q3 = df['closing_price'].quantile(0.75)
45 IQR = Q3 - Q1
46
47 lower_limit = Q1 - 1.5 * IQR
48 upper_limit = Q3 + 1.5 * IQR
49
50 # Mark outliers
51 df['is_outlier'] = (df['closing_price'] < lower_limit) | (df['closing_price'] > upper_limit)
52
53 # -----
54 # Step 6: Display final preprocessed dataset
55 # -----
56 print("✅ Financial Data Preprocessing Complete\n")
57 print(df)
58

```

OUTPUT :

```

PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi priya/Desktop/ai lab/ASSIGNMENT2.py"
c:/Users/keerthi priya/Desktop/ai lab/ASSIGNMENT2.py:33: FutureWarning: The default fill_method='pad' in Series.pct_change is deprecated and will be removed in a future version.
Either fill in any non-leading NA values prior to calling pct_change or specify 'fill_method=None' to not fill NA values.
df['return_7d'] = df['closing_price'].pct_change(7)
✅ Financial Data Preprocessing Complete

   date      closing_price  volume  return_7d  volume_log  is_outlier
0 2025-10-01           150.0    1000.0        NaN     6.908755        False
1 2025-10-02           152.0    1050.0        NaN     6.957497        False
2 2025-10-03           NaN     980.0        NaN     6.888572        False
3 2025-10-04           155.0    1570.0        NaN     7.359468        False
4 2025-10-05           160.0    1200.0        NaN     7.090910        False
5 2025-10-06           300.0    5000.0        NaN     8.517393         True
6 2025-10-07           162.0    1150.0        NaN     7.048386        False
7 2025-10-08           158.0    1570.0    0.053333     7.359468        False
8 2025-10-09           NaN    1100.0    0.039474     7.003974        False
9 2025-10-10           159.0    1080.0    0.046053     6.985642        False
PS C:\Users\keerthi priya\Desktop\ai lab>

```

OBSERVATION :

1. Imported pandas and numpy for data handling and calculations.
2. Created a sample stock dataset with date, closing price, and volume.
3. Filled missing closing_price using fillna and volume with mean value.
4. Applied log normalization to the volume column to reduce skewness.
5. Added a new column is_outlier showing whether a price value is unusually high/low.
6. Final dataset is clean, normalized, and feature-rich for stock analysis or modelling

#TASK-3

PROMPT :

Write a python program to Clean and preprocess IoT temperature and humidity logs.

Instructions:

- Handle missing values using forward fill.
- Remove sensor drift (apply rolling mean).
- Normalize readings using standard scaling. - Encode categorical sensor IDs.

CODE :

```
assignment3.py > ...
1  import os
2  import pandas as pd
3  from sklearn.preprocessing import StandardScaler, LabelEncoder
4
5  # -----
6  # Step 1: Check if dataset exists, else create it
7  # -----
8  file_path = "iot_sensor_logs.csv"
9
10 if not os.path.exists(file_path):
11     data = {
12         "timestamp": [
13             "2025-10-27 10:00",
14             "2025-10-27 10:05",
15             "2025-10-27 10:10",
16             "2025-10-27 10:15",
17             "2025-10-27 10:20",
18             "2025-10-27 10:25"
19         ],
20         "sensor_id": ["S1", "S1", "S2", "S2", "S1", "S2"],
21         "temperature": [25.4, 26.1, 27.3, 27.8, None, 28.2],
22         "humidity": [60.2, 61.0, 63.1, 64.0, 65.0, None]
23     }
24     df = pd.DataFrame(data)
25     df.to_csv(file_path, index=False)
26     print(f"✅ Sample dataset '{file_path}' created!\n")
27
28 # -----
29 # Step 2: Load Dataset
30 # -----
31 df = pd.read_csv(file_path)
32
33 # -----
34 # Step 3: Handle Missing Values (Forward Fill)
35 # -----
36 df[['temperature', 'humidity']] = df[['temperature', 'humidity']].ffill()
37
```


Instructions:

- Standardize text (lowercase, remove HTML tags).
- Tokenize and encode reviews using AI-assisted methods (TF-IDF or embeddings).
- Handle missing ratings (fill with median).
- Normalize ratings (0–10 → 0–1 scale).
- Generate a before vs after summary report.

CODE :

```
assignment4.py > ...
1  import os
2  import re
3  import pandas as pd
4  from sklearn.feature_extraction.text import TfidfVectorizer
5  from sklearn.preprocessing import MinMaxScaler
6
7  # -----
8  # Step 1: Create sample dataset (if missing)
9  # -----
10 file_path = "movie_reviews.csv"
11
12 if not os.path.exists(file_path):
13     data = {
14         "review_id": [1, 2, 3, 4, 5],
15         "review_text": [
16             "<b>Excellent!</b> The movie was AMAZING 🍿",
17             "Good movie, but a bit lengthy.",
18             None,
19             "<i>Average</i> storyline, poor acting.",
20             "Worst movie ever! Waste of time..."
21         ],
22         "rating": [9.5, 8.0, None, 5.0, 2.0]
23     }
24     df = pd.DataFrame(data)
25     df.to_csv(file_path, index=False)
26     print(f"✅ Sample dataset '{file_path}' created!\n")
27
28 # -----
29 # Step 2: Load dataset
30 # -----
31 df = pd.read_csv(file_path)
32
33 print("📁 Before Cleaning:\n")
34 print(df, "\n")
35
36 # -----
37 # Step 3: Standardize Text
```



```
assignment4.py > ...
36 # -----
37 # Step 3: Standardize Text
38 #   - Convert to lowercase
39 #   - Remove HTML tags
40 # -----
41 def clean_text(text):
42     if pd.isna(text):
43         return ""
44     text = re.sub(r'<.*?>', '', text) # Remove HTML tags
45     return text.lower()
46
47 df['cleaned_review'] = df['review_text'].apply(clean_text)
48
49 # -----
50 # Step 4: Handle Missing Ratings (fill with median)
51 # -----
52 df['rating'] = df['rating'].fillna(df['rating'].median())
53
54 # -----
55 # Step 5: Normalize Ratings (0-10 → 0-1 scale)
56 # -----
57 scaler = MinMaxScaler(feature_range=(0, 1))
58 df['normalized_rating'] = scaler.fit_transform(df[['rating']])
59
60 # -----
61 # Step 6: Tokenize and Encode Reviews (TF-IDF)
62 # -----
63 vectorizer = TfidfVectorizer(stop_words='english', max_features=5)
64 tfidf_matrix = vectorizer.fit_transform(df['cleaned_review'])
65 tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=vectorizer.get_feature_names_out())
66
67 # Merge encoded features back to main DataFrame
68 df_final = pd.concat([df, tfidf_df], axis=1)
69
70 # -----
71 # Step 7: Generate Before vs After Summary
72 # -----
```

```

assignment4.py > ...
70 # -----
71 # Step 7: Generate Before vs After Summary
72 # -----
73 before_summary = {
74     "Missing Reviews": df['review_text'].isna().sum(),
75     "Missing Ratings": df['rating'].isna().sum(),
76     "Text Format": "Mixed case + HTML tags",
77     "Rating Range": "0-10"
78 }
79
80 after_summary = {
81     "Missing Reviews": df_final['cleaned_review'].isna().sum(),
82     "Missing Ratings": df_final['rating'].isna().sum(),
83     "Text Format": "Lowercase + Cleaned",
84     "Rating Range": "0-1 normalized"
85 }
86
87 # -----
88 # Step 8: Display Results
89 # -----
90 print("✅ After Cleaning & Encoding:\n")
91 print(df_final, "\n")
92
93 print("📄 Summary Report:")
94 print(pd.DataFrame([before_summary, after_summary], index=["Before", "After"]))

```

OUTPUT :

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\keerthi priya\Desktop\ai lab> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "C:/Users/keerthi priya/Desktop/ai lab/assignment4.py"
Before Cleaning:
  review_id  review_text  rating
0         1  <b>Excellent!</b> The movie was AMAZING 🍿  9.5
1         2    Good movie, but a bit lengthy.  8.0
2         3                NaN  NaN
3         4  <i>Average</i> storyline, poor acting.  5.0
4         5    Worst movie ever! Waste of time...  2.0

✅ After Cleaning & Encoding:
  review_id  review_text  rating  cleaned_review  normalized_rating  acting  amazing  average  bit  movie
0         1  <b>Excellent!</b> The movie was AMAZING 🍿  9.5  excellent! the movie was amazing 🍿  1.0  0.000000  0.830881  0.000000  0.000000  0.556451
1         2    Good movie, but a bit lengthy.  8.0  good movie, but a bit lengthy.  0.8  0.000000  0.000000  0.000000  0.830881  0.556451
2         3                NaN  6.5                NaN  0.6  0.000000  0.000000  0.000000  0.000000  0.000000
3         4  <i>Average</i> storyline, poor acting.  5.0  average storyline, poor acting.  0.4  0.707107  0.000000  0.707107  0.000000  0.000000
4         5    Worst movie ever! Waste of time...  2.0  worst movie ever! waste of time...  0.0  0.000000  0.000000  0.000000  0.000000  1.000000

📄 Summary Report:
  Missing Reviews  Missing Ratings  Text Format  Rating Range
Before           1              0  Mixed case + HTML tags  0-10
After            0              0  Lowercase + Cleaned  0-1 normalized

```

OBSERVATION :

1. All reviews were converted to lowercase for consistency.
2. HTML tags such as , <i> were successfully removed.
3. Missing review texts were replaced with empty strings ("").
4. Removed inconsistencies in text format and missing values.
5. The final dataset is clean, consistent, and AI-ready for sentiment or anomaly detection tasks.

